

crashed. More details on 3PC can be found in [Bernstein et al., 1987] and [Özsu and Valduriez, 2011].

8.6 Recovery

So far, we have mainly concentrated on algorithms that allow us to tolerate faults. However, once a failure has occurred, it is essential that the process where the failure happened can recover to a correct state. In what follows, we first concentrate on what it actually means to recover to a correct state, and subsequently when and how the state of a distributed system can be recorded and recovered to, by means of checkpointing and message logging.

Introduction

Fundamental to fault tolerance is the recovery from an error. Recall that an error is that part of a system that may lead to a failure. The whole idea of error recovery is to replace an erroneous state with an error-free state. There are essentially two forms of error recovery.

In **backward recovery**, the main issue is to bring the system from its present erroneous state back into a previously correct state. To do so, it will be necessary to record the system's state from time to time, and to restore such a recorded state when things go wrong. Each time (part of) the system's present state is recorded, a **checkpoint** is said to be made.

Another form of error recovery is **forward recovery**. In this case, when the system has entered an erroneous state, instead of moving back to a previous, checkpointed state, an attempt is made to bring the system in a correct new state from which it can continue to execute. The main problem with forward error recovery mechanisms is that it has to be known in advance which errors may occur. Only in that case is it possible to correct those errors and move to a new state.

The distinction between backward and forward error recovery is easily explained when considering the implementation of reliable communication. The common approach to recover from a lost packet is to let the sender retransmit that packet. In effect, packet retransmission establishes that we attempt to go back to a previous, correct state, namely the one in which the packet that was lost is being sent. Reliable communication through packet retransmission is therefore an example of applying backward error recovery techniques.

An alternative approach is to use a method known as **erasure correction**. In this approach, a missing packet is constructed from other, successfully delivered packets. For example, in an (n, k) -block erasure code, a set of k *source packets* is encoded into a set of n *encoded packets*, such that *any* set of k

encoded packets is enough to reconstruct the original k source packets. Typical values are $k = 16$ or $k = 32$, and $k < n \leq 2k$ (see, for example, Rizzo [1997]). If not enough packets have yet been delivered, the sender will have to continue transmitting packets until a previously lost packet can be constructed. Erasure correction is a typical example of a forward error recovery approach.

By and large, backward error recovery techniques are widely applied as a general mechanism for recovering from failures in distributed systems. The major benefit of backward error recovery is that it is a generally applicable method independent of any specific system or process. In other words, it can be integrated into (the middleware layer) of a distributed system as a general-purpose service.

However, backward error recovery also introduces some problems [Singhal and Shivaratri, 1994]. First, restoring a system or process to a previous state is generally a relatively costly operation in terms of performance. As will be discussed in succeeding sections, much work generally needs to be done to recover from, for example, a process crash or site failure. A potential way out of this problem, is to devise very cheap mechanisms by which components are simply rebooted.

Second, because backward error recovery mechanisms are independent of the distributed application for which they are actually used, no guarantees can be given that once recovery has taken place, the same or similar failure will not happen again. If such guarantees are needed, handling errors often requires that the application gets into the loop of recovery. In other words, full-fledged failure transparency can generally not be provided by backward error recovery mechanisms.

Finally, although backward error recovery requires checkpointing, some states can simply never be rolled back to. For example, once a (possibly malicious) person has taken the \$1,000 that suddenly came rolling out of the incorrectly functioning automated teller machine, there is only a small chance that money will be stuffed back in the machine. Likewise, recovering to a previous state in most Unix systems after having enthusiastically typed

```
/bin/rm -fr *
```

but from the wrong working directory, may turn a few people pale. Some things are simply irreversible.

Checkpointing allows the recovery to a previous correct state. However, taking a checkpoint is often a costly operation and may have a severe performance penalty. As a consequence, many fault-tolerant distributed systems combine checkpointing with **message logging**. In this case, after a checkpoint has been taken, a process logs its messages before sending them off (called **sender-based logging**). An alternative solution is to let the receiving process first log an incoming message before delivering it to the application it is executing. This scheme is also referred to as **receiver-based logging**. When a receiving process crashes, it is necessary to restore the most recently

checkpointed state, and from there on *replay* the messages that have been sent. Consequently, combining checkpoints with message logging makes it possible to restore a state that lies beyond the most recent checkpoint without the cost of checkpointing.

In a system where only checkpointing is used, processes will be restored to a checkpointed state. From there on, their behavior may be different than it was before the failure occurred. For example, because communication times are not deterministic, messages may now be delivered in a different order, in turn leading to different reactions by the receivers. However, if message logging takes place, an actual replay of the events that happened since the last checkpoint takes place. Such a replay makes it easier to interact with the outside world.

For example, consider the case that a failure occurred because a user provided erroneous input. If only checkpointing is used, the system would have to take a checkpoint before accepting the user's input in order to recover to exactly the same state. With message logging, an older checkpoint can be used, after which a replay of events can take place up to the point that the user should provide input. In practice, the combination of having fewer checkpoints and message logging is more efficient than having to take many checkpoints.

Elnozahy et al. [2002] provide a survey of checkpointing and logging in distributed systems. Various algorithmic details can be found in Chow and Johnson [1997].

Checkpointing

In a fault-tolerant distributed system, backward error recovery requires that the system regularly saves its state². In particular, we need to record a consistent global state, also called a **distributed snapshot**. In a distributed snapshot, if a process P has recorded the receipt of a message, then there should also be a process Q that has recorded the sending of that message. After all, it must have come from somewhere.

To recover after a process or system failure requires that we construct a consistent global state from local states as saved by each process. In particular, it is best to recover to the *most recent* distributed snapshot, also referred to as a **recovery line**. In other words, a recovery line corresponds to the most recent consistent collection of checkpoints, as shown in Figure 8.35.

Coordinated checkpointing

In **coordinated checkpointing** all processes synchronize to jointly write their state to local storage. The main advantage of coordinated checkpointing is

²We assume that each process has access to a local, reliable storage.

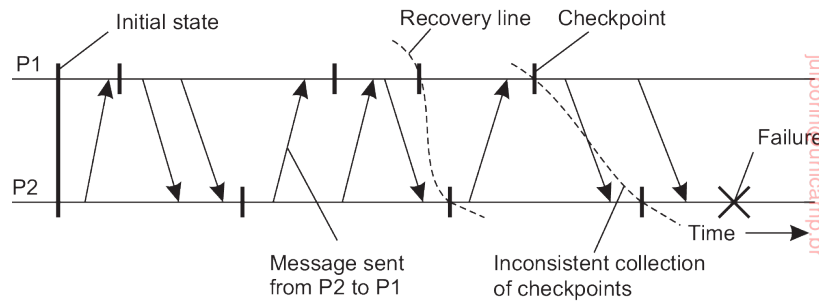


Figure 8.35: A recovery line.

that the saved state is automatically globally consistent. A simple solution is to use a two-phase blocking protocol. A coordinator first multicasts a CHECKPOINT-REQUEST message to all processes. When a process receives such a message, it takes a local checkpoint, queues any subsequent message handed to it by the application it is executing, and acknowledges to the coordinator that it has taken a checkpoint. When the coordinator has received an acknowledgment from all processes, it multicasts a CHECKPOINT-DONE message to allow the (blocked) processes to continue.

It is easy to see that this approach will also lead to a globally consistent state, because no incoming message will ever be registered as part of a checkpoint. The reason for this is that any message that follows a request for taking a checkpoint is not considered to be part of the local checkpoint. At the same time, outgoing messages (as handed to the checkpointing process by the application it is running), are queued locally until the CHECKPOINT-DONE message is received.

An improvement to this algorithm is to send a checkpoint request only to those processes that depend on the recovery of the coordinator, and ignore the other processes. A process is dependent on the coordinator if it has received a message that is directly or indirectly causally related to a message that the coordinator had sent since the last checkpoint. This leads to the notion of an **incremental snapshot**.

To take an incremental snapshot, the coordinator sends a checkpoint request only to those processes it had sent a message to since it last took a checkpoint. When a process P receives such a request, it forwards the request to all those processes to which P itself had sent a message since the last checkpoint, and so on. A process forwards the request only once. When all processes have been identified, a second multicast is used to actually trigger checkpointing and to let the processes continue where they had left off.

Independent checkpointing

Now consider the case in which each process simply records its local state from time to time in an uncoordinated fashion. To discover a recovery line requires that each process is rolled back to its most recently saved state. If these local states jointly do not form a distributed snapshot, further rolling back is necessary. This process of a cascaded rollback may lead to what is called the **domino effect** and is shown in Figure 8.36.

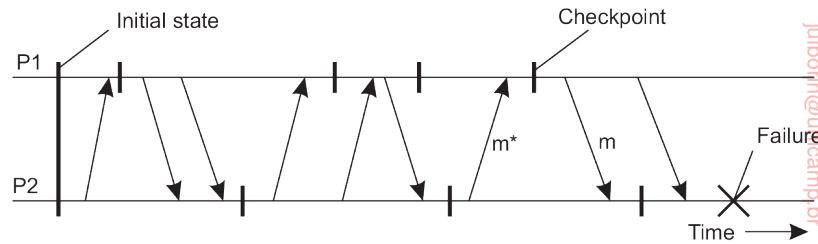


Figure 8.36: The domino effect.

When process P_2 crashes, we need to restore its state to the most recently saved checkpoint. As a consequence, process P_1 will also need to be rolled back. Unfortunately, the two most recently saved local states do not form a consistent global state: the state saved by P_2 indicates the receipt of a message m , but no other process can be identified as its sender. Consequently, P_2 needs to be rolled back to an earlier state.

However, the next state to which P_2 is rolled back also cannot be used as part of a distributed snapshot. In this case, P_1 will have recorded the receipt of message m^* , but there is no recorded event of this message being sent. It is therefore necessary to also roll P_1 back to a previous state. In this example, it turns out that the recovery line is actually the initial state of the system.

As processes take local checkpoints independent of each other, this method is also referred to as **independent checkpointing**. Its implementation requires that dependencies are recorded in such a way that processes can jointly roll back to a consistent global state. To that end, let $CP_i(m)$ denote the m -th checkpoint taken by process P_i . Also, let $INT_i(m)$ denote the interval between checkpoints $CP_i(m-1)$ and $CP_i(m)$.

When process P_i sends a message in interval $INT_i(m)$, it piggybacks the pair (i, m) to the receiving process. When process P_j receives a message in interval $INT_j(n)$, along with the pair of indices (i, m) , it records the dependency $INT_i(m) \rightarrow INT_j(n)$. Whenever P_j takes checkpoint $CP_j(n)$, it additionally saves this dependency to its local storage, along with the rest of the recovery information that is part of $CP_j(n)$.

Now suppose that at a certain moment, process P_1 is required to roll back to checkpoint $CP_i(m-1)$. To ensure global consistency, we need to ensure that all processes that have received messages from P_i and that were sent in

interval $INT_i(m)$, are rolled back to a checkpointed state preceding the receipt of such messages. In particular, process P_j in our example, will need to be rolled back at least to checkpoint $CP_j(n-1)$. If $CP_j(n-1)$ does not lead to a globally consistent state, further rolling back may be necessary.

Calculating the recovery line requires an analysis of the interval dependencies recorded by each process when a checkpoint was taken. Without going into any further details, it turns out that such calculations are fairly complex. In addition, as it turns out, it is often not the coordination between processes that is the dominating performance factor, but the overhead as the result of having to save the state to local stable storage. Therefore, coordinated checkpointing, which is much simpler than independent checkpointing, is often more popular, and will presumably stay so even when systems grow to much larger sizes [Elnozahy and Plank, 2004]

Message logging

Considering that checkpointing can be an expensive operation, techniques have been sought to reduce the number of checkpoints, but still enable recovery. One such technique is logging messages. The basic idea underlying message logging is that if the transmission of messages can be *replayed*, we can still reach a globally consistent state but without having to restore that state from local storages. Instead, a checkpointed state is taken as a starting point, and all messages that have been sent since are simply retransmitted and handled accordingly.

This approach works fine under the assumption of what is called a **piecewise deterministic execution model**. In such a model, the execution of each process is assumed to take place as a series of intervals in which events take place. These events are the same as those discussed in the context of Lamport's happened-before relationship in Chapter 6. For example, an event may be the execution of an instruction, the sending of a message, and so on. Each interval in the piecewise deterministic model is assumed to start with a nondeterministic event, such as the receipt of a message. However, from that moment on, the execution of the process is completely deterministic. An interval ends with the last event before a nondeterministic event occurs.

In effect, an interval can be replayed with a known result, that is, in a completely deterministic way, provided it is replayed starting with the same nondeterministic event as before. Consequently, if we record all nondeterministic events in such a model, it becomes possible to completely replay the entire execution of a process in a deterministic way.

Considering that message logs are necessary to recover from a process crash so that a globally consistent state is restored, it becomes important to know precisely when messages are to be logged. Following the approach described by Alvisi and Marzullo [1998], it turns out that many existing

message-logging schemes can be easily characterized if we concentrate on how they deal with orphan processes.

An **orphan process** is a process that survives the crash of another process, but whose state is inconsistent with the crashed process after its recovery. As an example, consider the situation shown in Figure 8.37. Process Q receives messages m_1 and m_2 from process P and R, respectively, and subsequently sends a message m_3 to R. However, in contrast to all other messages, message m_2 is not logged. If process Q crashes and later recovers again, only the logged messages required for the recovery of Q are replayed, in our example, m_1 . Because m_2 was not logged, its transmission will not be replayed, meaning that the transmission of m_3 may also not take place.

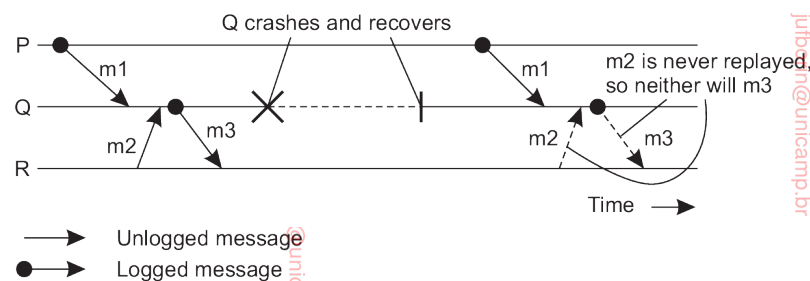


Figure 8.37: Incorrect replay of messages after recovery, leading to an orphan process R.

However, the situation after the recovery of Q is inconsistent with that before its recovery. In particular, R holds a message (m_3) that was sent before the crash, but whose receipt and delivery do not take place when replaying what had happened before the crash. Such inconsistencies should obviously be avoided.

Note 8.14 (Advanced: Characterizing message-logging schemes)

To characterize different message-logging schemes, we follow the approach described by Alvisi and Marzullo [1998]. Each message m is considered to have a header that contains all information necessary to retransmit m , and to properly handle it. For example, each header will identify the sender and the receiver, but also a sequence number to recognize it as a duplicate. In addition, a delivery number may be added to decide when exactly it should be handed over to the receiving application.

A message is said to be **stable** if it can no longer be lost, for example, because it has been written to reliable, local storage. Stable messages can thus be used for recovery by replaying their transmission.

Each message m leads to a set $\text{DEP}(m)$ of processes that depend on the delivery of m . In particular, $\text{DEP}(m)$ consists of those processes to which m has

been delivered. In addition, if another message m^* is causally dependent on the delivery of m , and m^* has been delivered to a process Q , then Q will also be contained in $\text{DEP}(m)$. Note that m^* is causally dependent on the delivery of m , if it was sent by the same process that previously delivered m , or which had delivered another message that was causally dependent on the delivery of m .

The set $\text{COPY}(m)$ consists of those processes that have a copy of m , but have not (yet) reliably stored it. When a process Q delivers message m , it also becomes a member of $\text{COPY}(m)$. Note that $\text{COPY}(m)$ consists of those processes that could hand over a copy of m that can be used to replay the transmission of m . If all these processes crash, replaying the transmission of m is clearly not feasible.

Using these notations, it is now easy to define precisely what an orphan process is. Let FAIL denote the collection of crashed processes, and assume Q is one of the survivors. Q is an orphan process if there is a message m , such that Q is contained in $\text{DEP}(m)$, while at the same time every process in $\text{COPY}(m)$ has crashed. More formally:

$$Q \text{ is orphaned} \Leftrightarrow \exists m : Q \in \text{DEP}(m) \text{ and } \text{COPY}(m) \subseteq \text{FAIL}$$

In other words, an orphan process appears when it is dependent on m , but there is no way to replay m 's transmission.

To avoid orphan processes, we thus need to ensure that if each process in $\text{COPY}(m)$ crashed, then no surviving process is left in $\text{DEP}(m)$. In other words, all processes in $\text{DEP}(m)$ should have crashed as well. This condition can be enforced if we can guarantee that whenever a process becomes a member of $\text{DEP}(m)$, it also becomes a member of $\text{COPY}(m)$. In other words, whenever a process becomes dependent on the delivery of m , it will keep a copy of m .

There are essentially two approaches that can now be followed. The first approach is represented by what are called **pessimistic logging protocols**. These protocols take care that for each *nonstable* message m , there is at most one process dependent on m . In other words, pessimistic logging protocols ensure that each nonstable message m is delivered to at most one process. Note that as soon as m is delivered to, say process P , P becomes a member of $\text{COPY}(m)$.

The worst that can happen is that process P crashes without m ever having been logged. With pessimistic logging, P is not allowed to send any messages after the delivery of m without first having ensured that m has been written to reliable storage. Consequently, no other processes will ever become dependent on the delivery of m to P , without having the possibility of replaying the transmission of m . In this way, orphan processes are always avoided.

In contrast, in an **optimistic logging protocol**, the actual work is done *after* a crash occurs. In particular, assume that for some message m , each process in $\text{COPY}(m)$ has crashed. In an optimistic approach, any orphan process in $\text{DEP}(m)$ is rolled back to a state in which it no longer belongs to $\text{DEP}(m)$. Clearly, optimistic logging protocols need to keep track of dependencies, which complicates their implementation.

As pointed out by Elnozahy et al. [2002], pessimistic logging is so much simpler than optimistic approaches, that it is the preferred way of message logging in practical distributed systems design.

Recovery-oriented computing

A related way of handling recovery is essentially to start over again. The underlying principle toward this way of masking failures is that it may be much cheaper to optimize for recovery, then it is aiming for systems that are free from failures for a long time. This approach is also referred to as **recovery-oriented computing** [Candea et al., 2004a].

There are different flavors of recovery-oriented computing. One flavor is to simply reboot (part of a system) and has been explored to restart Internet servers [Candea et al., 2004b; 2006]. In order to be able to reboot only a part of the system, it is crucial the fault is properly localized. At that point, rebooting simply means deleting all instances of the identified components, along with the threads operating on them, and (often) to just restart the associated requests. Note that fault localization itself may be a nontrivial exercise [Steinder and Sethi, 2004].

To enable rebooting as a practical recovery technique requires that components are largely decoupled in the sense that there are few or no dependencies between different components. If there are strong dependencies, then fault localization and analysis may still require that a complete server needs to be restarted at which point applying traditional recovery techniques as the ones we just discussed may be more efficient.

Another flavor of recovery-oriented computing is to apply checkpointing and recovery techniques, but to continue execution in a changed environment. The basic idea here is that many failures can be simply avoided if programs are given some more buffer space, memory is zeroed before allocated, changing the ordering of message delivery (as long as this does not affect semantics), and so on [Qin et al., 2005]. The key idea is to tackle software failures (whereas many of the techniques discussed so far are aimed at, or are based on hardware failures). Because software execution is highly deterministic, changing an execution environment may save the day, but, of course, without repairing anything.

8.7 Summary

Fault tolerance is an important subject in distributed systems design. Fault tolerance is defined as the characteristic by which a system can mask the occurrence and recovery from failures. In other words, a system is fault tolerant if it can continue to operate in the presence of failures.

Several types of failures exist. A crash failure occurs when a process simply halts. An omission failure occurs when a process does not respond to incoming requests. When a process responds too soon or too late to a request, it is said to exhibit a timing failure. Responding to an incoming request, but in the wrong way, is an example of a response failure. The most difficult failures

to handle are those by which a process exhibits any kind of failure, called arbitrary or Byzantine failures.

Redundancy is the key technique needed to achieve fault tolerance. When applied to processes, the notion of process groups becomes important. A process group consists of a number of processes that closely cooperate to provide a service. In fault-tolerant process groups, one or more processes can fail without affecting the availability of the service the group implements. Often, it is necessary that communication within the group be highly reliable, and adheres to stringent ordering and atomicity properties in order to achieve fault tolerance.

The real problem is that members of a process group need to reach consensus in the presence of various failures. Paxos is by now a well-established and highly robust consensus algorithm. By using $2k + 1$ servers, it can establish k -fault tolerance. However, we need a total of $3k + 1$ servers if it is needed to deal with arbitrary failures.

Reliable group communication, also called reliable multicasting, comes in different forms. As long as groups are relatively small, it turns out that implementing reliability is feasible. However, as soon as very large groups need to be supported, scalability of reliable multicasting becomes problematic. The key issue in achieving scalability is to reduce the number of feedback messages by which receivers report the (un)successful receipt of a multicasted message.

Matters become worse when atomicity is to be provided. In atomic multicast protocols, it is essential that each group member has the same view concerning to which members a multicasted message has been delivered. Atomic multicasting can be precisely formulated in terms of a virtual synchronous execution model. In essence, this model introduces boundaries between which group membership does not change and which messages are reliably transmitted. A message can never cross a boundary.

Group membership changes are an example where each process needs to agree on the same list of members. Such agreement can be reached by means of a commit protocol, of which the two-phase commit protocol is the most widely applied. In a two-phase commit protocol, a coordinator first checks whether all processes agree to perform the same operation (i.e., whether they all agree to commit), and in a second round, multicasts the outcome of that poll. A three-phase commit protocol is used to handle the crash of the coordinator without having to block all processes to reach agreement until the coordinator recovers.

Recovery in fault-tolerant systems is invariably achieved by checkpointing the state of the system on a regular basis. Checkpointing is completely distributed. Unfortunately, taking a checkpoint is an expensive operation. To improve performance, many distributed systems combine checkpointing with message logging. By logging the communication between processes,

it becomes possible to replay the execution of the system after a crash has occurred.